МОСКОВСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
им. Н.Э. Баумана

Факультет “Инженерный бизнес и менеджмент”
Кафедра “Промышленная логистика (Бизнес-информатика)”



Дисциплина “Парадигмы и конструкции языков программирования”

Отчет по лабораторной работе №2

Выполнил:
Дмитриева В. А.

Преподаватель:

Правдина А. Д.

Бурмистрова М. В.

Задание

1. Необходимо создать виртуальное окружение и установить в него хотя бы один внешний пакет с использованием pip.

2. Необходимо разработать программу, реализующую работу с классами. Программа должна быть разработана в виде консольного приложения на языке **Python 3**.

3. Все файлы проекта (кроме основного файла main.py) должны располагаться в пакете lab_python_oop.

4. Каждый из нижеперечисленных классов должен располагаться в **отдельном файле** пакета lab_python_oop.

Вариант 11

Написать программу, в которой описана иерархия классов: функция от одной переменной (арксинус, арккосинус, а также класс, необходимый для представления производных). Базовый класс должен иметь методы получения значения функции для данного значения переменной, а также создания экземпляра класса, представляющего собой производную текущего экземпляра. Продемонстрировать работу всех методов классов всех классов.

- **Абстрактный класс** *Подробнее про абстрактные классы и методы Вы можете прочитать здесь.*
- **Классы** *Подробнее про описание свойств Вы можете прочитать здесь.*

Требования к классам

1. На основании предложенной предметной области спроектировать классы, используя механизм наследования. Для каждого класса использовать отдельный модуль, можно добавить свой клас в иерархию.
2. Предусмотреть у класса наличие полей, методов и свойств. Названия членов класса должны быть осмысленны и снабжены комментариями. Все атрибуты экземпляров описать как приватные (private), добавив для каждого декоратор (property) (см. ПРИЛОЖЕНИЕ. Пример использования объектов property). В сеттерах учесть ограничения, накладываемые на данные.
3. Один из наследников должен перегружать метод родителя.
4. Один из классов должен содержать метод, который переопределяется в одном наследнике и не переопределяется в другом.
5. Если для экземпляра класса будут создаваться атрибуты, то определить в классе конструктор и метод **str**; в демонстрации использовать вызов функции print для

экземпляра класса. В конструкторах подклассов вызывать конструкторы суперклассов: super.**init**().

6. Продемонстрировать работу всех объявленных методов, предоставив пользователю выбор типа объекта для демонстрации.

Файл main.py

В корневом каталоге проекта создайте файл main.py для тестирования Ваших классов (используйте следующую конструкцию — https://docs.python.org/3/library/**main**.html).

Создайте несколько объектов Ваших классов, демонстрирующих работу всех реализованных методов, и выведите о них информацию в консоль (**N** — номер Вашего варианта по списку группы).

Также вызовите один из методов внешнего пакета, установленного с использованием pip.

Текст программы

Файл function.py (абстрактный класс)

```
from abc import ABC, abstractmethod

class Function(ABC):
    """Абстрактная функция одной переменной: f(x) = k * g(x)"""

    def __init__(self, name, coefficient=1.0):
        self.__name = ""
        self.__coefficient = 1.0
        self.name = name
        self.coefficient = coefficient

    @property
    def name(self):
        """Название функции"""
        return self.__name

    @name.setter
    def name(self, value):
        if not isinstance(value, str):
            raise TypeError("Название должно быть строкой")
        if not value.strip():
            raise ValueError("Название не может быть пустым")
        self.__name = value.strip()

    @property
    def coefficient(self):
```

```
        """Множитель k перед функцией"""
        return self.__coefficient

    @coefficient.setter
    def coefficient(self, value):
        if isinstance(value, bool) or not isinstance(value, (int, float)):
            raise TypeError("Коэффициент должен быть числом")
        if value == 0:
            raise ValueError("Коэффициент не может быть равен нулю")
        self.__coefficient = float(value)

    @staticmethod
    def check_x(x):
        """Проверка, что аргумент - число"""
        if isinstance(x, bool) or not isinstance(x, (int, float)):
            raise TypeError("Аргумент x должен быть числом")

    @abstractmethod
    def value(self, x):
        """Возвращает значение функции в точке x"""

    @abstractmethod
    def derivative(self):
        """Возвращает производную функции"""

    def describe(self):       #в Arcsin переопределяется, в Arccos нет
        """Общее описание"""
        return f"Тригонометрическая функция '{self.name}' вида k*g(x), k =
{self.coefficient}"

    def __str__(self):
        return f"{self.name} с k = {self.coefficient}"
```

Файл derivative.py (наследник)

```
import math
from lab_python_oop.function import Function

class Derivative(Function):
    """Производная функции: f'(x) = k / sqrt(1 - x^2)"""

    def __init__(self, source, factor):
        super().__init__("Производная от " + source.name, factor)
        self.__source = source      # исходная функция

    @property
    def source(self):
        """Функция, от которой взята производная"""
        return self.__source
```

```
    def value(self, x):
        self.check_x(x)
        if not -1 < x < 1:
            raise ValueError("Производная определена только при -1 < x < 1")
        return self.coefficient / math.sqrt(1 - x ** 2)

    def derivative(self):
        raise NotImplementedError("Вторая производная не реализована")

    def __str__(self):
        return super().__str__()
```

Файл arccos.py (наследник)

```
import math
from lab_python_oop.function import Function
from lab_python_oop.derivative import Derivative

class Arccos(Function):
    """f(x) = k * arccos(x), область определения [-1; 1]"""

    def __init__(self, coefficient=1.0):
        super().__init__("Арккосинус", coefficient)

    def value(self, x):
        self.check_x(x)
        if not -1 <= x <= 1:
            raise ValueError("Арккосинус определён только при -1 <= x <= 1")
        return self.coefficient * math.acos(x)

    def derivative(self):
        return Derivative(self, -self.coefficient)     # -k / sqrt(1 - x^2)

    def __str__(self):        # расширяем метод родителя через super()
        return super().__str__() + ", область определения [-1; 1], область значений
[0; pi]"
```

Файл arcsin.py (наследник)

```
import math
from lab_python_oop.function import Function
from lab_python_oop.derivative import Derivative

class Arcsin(Function):
    """f(x) = k * arcsin(x), область определения [-1; 1]"""

    def __init__(self, coefficient=1.0):
        super().__init__("Арксинус", coefficient)

    def value(self, x):
```

```
        self.check_x(x)
        if not -1 <= x <= 1:
            raise ValueError("Арксинус определён только при -1 <= x <= 1")
        return self.coefficient * math.asin(x)

    def derivative(self):
        return Derivative(self, self.coefficient)      # k / sqrt(1 - x^2)

    def describe(self):       #переопределяем метод родителя
        return "Арксинус: область определения [-1; 1], область значений [-pi/2;
pi/2]"
```

Файл main.py

```
import cowsay
from lab_python_oop.arcsin import Arcsin
from lab_python_oop.arccos import Arccos

def read_number(prompt):
    """Просит ввести число, пока пользователь не введёт корректно"""
    while True:
        try:
            return float(input(prompt))
        except ValueError:
            print("Нужно ввести число!")

def demo(func, x):
    """Показывает работу всех методов для объекта func в точке x"""
    print(func)
    print(func.describe())
    try:
        print(f"f({x}) = {func.value(x)}")
    except ValueError as e:
        print("Ошибка значения:", e)
    d = func.derivative()
    print(d)
    print(d.describe())
    print("Исходная функция производной:", d.source.name)
    try:
        print(f"f'({x}) = {d.value(x)}")
    except ValueError as e:
        print("Ошибка производной:", e)
    try:
        d.derivative()
    except NotImplementedError as e:
        print("Ожидаемая ошибка:", e)
    try:
        func.coefficient = 0
    except ValueError as e:
        print("Сеттер не пропустил:", e)
```

```
    try:
        func.name = "   "
    except ValueError as e:
        print("Сеттер не пропустил:", e)

def main():
    while True:
        print("\n1 - Арксинус\n2 - Арккосинус\n0 - Выход")
        choice = input("Выбор: ")
        if choice == "0":
            break
        if choice not in ("1", "2"):
            print("Нет такого пункта")
            continue
        k = read_number("Коэффициент k (не 0): ")
        x = read_number("Значение x (от -1 до 1): ")
        try:
            func = Arcsin(k) if choice == "1" else Arccos(k)
        except ValueError as e:
            print("Ошибка:", e)
            continue
        demo(func, x)
    cowsay.cow("Лабораторная готова!")           # метод внешнего пакета

if __name__ == "__main__":
    main()
```

Результаты выполнения программы

```
1 - Арксинус
2 - Арккосинус
0 - Выход
Выбор: 1
Коэффициент k (не 0): 3
Значение x (от -1 до 1): 0.5
Арксинус с k = 3.0
Арксинус: область определения [-1; 1], область значений [-pi/2; pi/2]
f(0.5) = 1.5707963267948968
Производная от Арксинус с k = 3.0
Тригонометрическая функция 'Производная от Арксинус' вида k*g(x), k = 3.0
Исходная функция производной: Арксинус
f'(0.5) = 3.464101615137755
Ожидаемая ошибка: Вторая производная не реализована
Сеттер не пропустил: Коэффициент не может быть равен нулю
Сеттер не пропустил: Название не может быть пустым
```

```
1 - Арксинус
2 - Арккосинус
0 - Выход
Выбор: 2
Коэффициент k (не 0): 4
Значение x (от -1 до 1): -0.5
Арккосинус с k = 4.0, область определения [-1; 1], область значений [0; pi]
Тригонометрическая функция 'Арккосинус' вида k*g(x), k = 4.0
f(-0.5) = 8.377580409572783
Производная от Арккосинус с k = -4.0
Тригонометрическая функция 'Производная от Арккосинус' вида k*g(x), k = -4.0
Исходная функция производной: Арккосинус
f'(-0.5) = -4.618802153517007
Ожидаемая ошибка: Вторая производная не реализована
Сеттер не пропустил: Коэффициент не может быть равен нулю
Сеттер не пропустил: Название не может быть пустым
```

```
1 - Арксинус
2 - Арккосинус
0 - Выход
Выбор: 0

  ____________________
| Лабораторная готова! |
  ====================
                     \
                      \
                        ^__^
                        (oo)\_______
                        (__)\       )\/\
                            ||----w |
                            ||     ||
(.venv) asus@DESKTOP-006N2OU:~/labs$
```